ECE
ÉCOLE D'INGÉNIEURS
ENGINEERING SCHOOL

Cours

Bases de données

Cours 5
Le Langage SQL (Partie 2)

Requêtes simples et agrégation

- Niveau: B1
- Année universitaire: 2025-2026

Dr. Salma Sassi

salma.sassi@ece.fr

Nous avons appris

- **créer** nos tables (langage DDL)
- **manipuler** leur contenu avec des opérations d'**insertion, de mise à jour et de suppression** (langage DML : INSERT, UPDATE, DELETE).

Nous allons apprendre à **interroger** notre base de données, c'est-à-dire **à extraire et croiser l'information** qu'elle contient

Objectifs de ce cours

A la fin de ce cours, vous saurez :

- Sélectionner des données avec SELECT et WHERE
- Trier et filtrer les résultats (ORDER BY, DISTINCT)
- Regrouper et agréger les données (GROUP BY, HAVING)

Exemple fil rouge du cours

Nous avons deux tables liées

Table 1: Département

dep_num	dep_nom
01	Ain
75	Paris
77	Seine-et-Marne
92	Hauts-de-Seine
93	Seine-Saint-Denis

Table 2: Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

Toutes nos requêtes SQL s'appuieront sur ces deux tables.

Requêtes de base

Afficher les informations principales des étudiants.

Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT nom, prenom, note
FROM Etudiant;
```

Résultat de la requête

nom	prenom	note
Martin	Véra	13.5
Martin	Annie	15.5
Dupont	Sylvie	15.0
Salma	Sassi	05.7
Dupond	Laurent	
Lefèvre	Laurent	11.5

SELECT précise **les colonnes** que l'on veut afficher dans le résultat.

- Ici : le nom, le prénom et la note de chaque étudiant.

FROM indique **la table** dans laquelle on va chercher les informations.

- Ici : la table ETUDIANT.

Renommer les colonnes du résultat (alias)

Afficher les étudiants avec des intitulés de colonnes plus clairs.

Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT nom AS "Nom de famille",
       prenom AS "Prénom de l'étudiant",
       note AS "Note finale"
FROM Etudiant;
```

Résultat de la requête

Nom de famille	Prénom de l'étudiant	Note finale
Martin	Véra	13.5
Martin	Annie	15.5
Dupont	Sylvie	15.0
Salma	Sassi	05.7
Dupond	Laurent	
Lefèvre	Laurent	11.5

L'ALIAS AS

- **Renommer temporairement** une ou plusieurs colonnes dans le résultat d'une requête.
- Faciliter la **lecture** et la **compréhension** des données affichées.
- Ne pas affecter le nom des colonnes dans la table d'origine.

Filtrer selon une condition (WHERE)

Afficher le nom et le prénom des étudiants inscrits dans le département numéro 77

Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

SELECT précise **les colonnes** que l'on veut afficher dans le résultat.

- Ici : le nom, le prénom et la note de chaque étudiant.

FROM indique **la table** dans laquelle on va chercher les informations.

- Ici : la table ETUDIANT.

La clause WHERE définit **la condition de filtrage** sur les lignes de la table.

- Ici : on ne garde que les étudiants inscrits dans le département numéro 77

```
SELECT nom, prenom, note, dep
FROM ETUDIANT
WHERE dep = '77';
```

Résultat de la requête

nom	prenom	note	dep
Martin	Véra	13.5	77
Dupont	Sylvie	15.0	77

Filtrer selon une condition (WHERE)

Afficher les étudiants inscrits en licence 2.

Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT nom, prenom, licence
FROM Etudiant
WHERE licence = '2';
```

Résultat de la requête

nom	prenom	licence
Martin	Véra	2
Dupont	Sylvie	2
Dupond	Laurent	2

La clause WHERE

- définit **la condition de filtrage** sur les lignes de la table.
  - Ici : on ne garde que les étudiants inscrits à la licence 2

Filtrer selon une condition (WHERE)

Afficher les étudiants de la licence 2 avec une note ≥ 12.

Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT nom, prenom, licence, note
FROM Etudiant
WHERE licence = '2' AND note >= 12;
```

Résultat de la requête

nom	prénom	Licence	Note
Martin	Véra	2	13.5
Dupont	Sylvie	2	15.0

LA CLAUSE WHERE

On peut **combiner** des conditions avec les opérateurs logiques :

- AND → toutes les conditions doivent être vraies.
- OR → au moins une condition doit être vraie.
- NOT → inverse le résultat d'une condition.

Trier les résultats (ORDER BY)

Afficher les étudiants triés par leur note décroissante

Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT nom, prenom, note
FROM Etudiant
ORDER BY note DESC;
```

Résultat de la requête

nom	prenom	note
Martin	Annie	15.5
Dupont	Sylvie	15.0
Martin	Véra	13.5
Lefèvre	Laurent	11.5
Salma	Sassi	5.7

La Clause ORDER BY

- **Trier les lignes du résultat d'une requête** selon une ou plusieurs colonnes.
- Par défaut, le tri est **croissant** (ASC).
- On peut préciser un tri **décroissant** avec le mot-clé **DESC**.

Comparer des textes avec LIKE

Afficher les numéros des départements dont le numéro commence par 7

Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT dep
FROM Etudiant
WHERE dep LIKE '7%';
```

Résultat de la requête

dep_num
77
75
77

La Clause LIKE avec (%)

- **Rechercher un motif (pattern)** dans une chaîne de caractères.
- Elle est utilisée pour **comparer partiellement des textes** à l'aide de symboles spéciaux :
  - **%** remplace **n'importe quelle suite de caractères** (y compris vide).

Comparer des textes avec LIKE

Afficher les Etudiants dont le nom commence par 'M' et comporte 6 lettres au total.

Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT nom, prenom
FROM Etudiant
WHERE nom LIKE 'M_ _ _ _ _';
```

Résultat de la requête

nom	prenom
Martin	Véra
Martin	Annie

La Clause LIKE avec (_)

- Le caractère _ remplace **exactement un caractère**.
- Ici, 'M_ _ _ _ _' signifie :
  ➜ un mot commençant par P et suivi de **5 lettres**.

Utiliser des opérateurs de comparaison

Afficher les étudiants ayant une note strictement supérieure à 15.

Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT nom, prenom, note
FROM Etudiant
WHERE note > 15;
```

Résultat de la requête

nom	prenom	note
Dupont	Sylvie	15.5

Les opérateurs de comparaison

Comparer des valeurs dans une condition

- > : supérieur à
- < : inférieur à
- >=, <=, = ou <> : opérateurs de comparaison

Utiliser BETWEEN pour une plage de valeurs

Afficher les étudiants ayant une note entre 10 et 15.

Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT nom, prenom, note
FROM Etudiant
WHERE note BETWEEN 10 AND 15;
```

Résultat de la requête

nom	prenom	note
Martin	Véra	13.5
Dupont	Sylvie	15.0
Lefèvre	Laurent	11.5

La clause BETWEEN

Simplifier les comparaisons d'intervalles numériques ou de dates.

- BETWEEN a AND b inclut les bornes a et b.
- Équivaut à : note >= 10 AND note <= 15.

Utiliser IN pour plusieurs valeurs

Afficher les étudiants du département 75 ou 92.

Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT nom, prenom, dep
FROM Etudiant
WHERE dep IN (75, 92);
```

Résultat de la requête

nom	prenom	dep
Martin	Annie	75
Dupond	Laurent	92

La clause IN

- Vérifier si une valeur fait partie d'un **ensemble de valeurs** précis.
- Simplifier l'écriture de plusieurs conditions en évitant de multiplier les OR.

Utiliser IS NULL / IS NOT NULL

Afficher les étudiants dont la note n'a pas encore été saisie.

Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT nom, prenom, note
FROM Etudiant
WHERE note IS NULL;
```

Résultat de la requête

nom	prenom	note
Dupond	Laurent	

IS NULL/IS NOT NULL

Détecter les valeurs manquantes

- IS NULL teste si une valeur est absente.
- IS NOT NULL fait l'inverse.

Opérateurs arithmétiques

Afficher la note sur 10.

Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT nom, prenom, note / 2 AS "Note sur 10"
FROM Etudiant;
```

Résultat de la requête

nom	prenom	**Note sur 10**
Martin	Véra	6,75
Martin	Annie	7,75
Dupont	Sylvie	7,5
Salma	Sassi	2,85
Dupond	Laurent	
Lefèvre	Laurent	5,75

Effectuer des calculs dans une requête

- Réaliser des **calculs directement dans une requête SQL** à l'aide des opérateurs arithmétiques : **+, -, *, /**.

- **Attention :**
  Si une valeur utilisée dans le calcul est **NULL**, le résultat de l'opération sera **NULL** également.

Concaténer du texte

Afficher le nom complet des étudiants.

Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT CONCAT(Nom, ' ', Prenom) AS "Nom complet"
FROM Etudiant;
```

Résultat de la requête

Nom complet
Martin Véra
Martin Annie
Dupont Sylvie
Salma Sassi
Dupond Laurent
Lefèvre Laurent

Fusionner plusieurs champs texte

- CONCAT() est la fonction de **concaténation de chaînes**.
- On peut ajouter des espaces ou du texte fixe entre guillemets.

Éliminer les doublons avec DISTINCT

Afficher la liste des départements sans répétition

Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT DISTINCT dep
FROM Etudiant;
```

Résultat de la requête

dep
77
75
93
92
NULL

La Clause DISTINCT

- **Supprimer les doublons** dans le résultat d'une requête.
- S'écrit juste après **SELECT** et conserve **une seule occurrence** de chaque valeur.
- Les valeurs **NULL** sont considérées comme identiques :
  - une seule sera affichée si plusieurs sont présentes.

Comparer des dates

Afficher les étudiants nés après le 1er janvier 2000.

Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT nom, prenom, date_n
FROM Etudiant
WHERE date_n > '1985-01-01';
```

Résultat de la requête

nom	prenom	date_n
Martin	Véra	1985-10-31
Martin	Annie	1985-12-31

Comparer les dates

- Les dates s'écrivent au format **AAAA-MM-JJ**.
- On peut utiliser les opérateurs <, >, <=, >=.
- SQL compare les dates **chronologiquement**.

Utiliser BETWEEN pour un intervalle

Afficher les étudiants nés entre 1985 et 1990.

Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT nom, prenom, date_n
FROM Etudiant
WHERE date_n BETWEEN '1985-01-01' AND '1990-12-31';
```

Résultat de la requête

nom	prenom	date_n
Martin	Véra	1985-10-31
Martin	Annie	1985-12-31

BETWEEN

- **BETWEEN** teste si une valeur est **dans un intervalle inclusif**.
- Les deux bornes sont **prises en compte**

Extraire des parties d'une date

Afficher les étudiants nés en 1985.

Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT nom, prenom, date_n
FROM Etudiant
WHERE YEAR(date_n) = 1985;
```

Résultat de la requête

nom	prenom	date_n
Martin	Véra	1985-10-31
Martin	Annie	1985-12-31

YEAR()

- la fonction **YEAR()** permet d'extraire une l'année d'une date. ➜ voir aussi MONTH() et DAY()
- Pratique pour filtrer selon **l'année, le mois ou le jour**.

Comparer avec la date du jour

Afficher les étudiants nés avant aujourd'hui.

Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT nom, prenom, date_n
FROM Etudiant
WHERE date_n < CURTIME;
```

Résultat de la requête

nom	prenom	date_n	note	dep	licence
Martin	Véra	1985-10-31	13.5	77	2
Martin	Annie	1985-12-31	15.5	75	1
Dupont	Sylvie	1983-02-03	15.0	77	2
Salma	Sassi	1983-10-22	05.7	93	1

CURTIME()

- **CURTIME** (ou CURRENT_DATE selon le SGBD) renvoie la **date du jour système**.
- Utile pour filtrer ou calculer des écarts temporels.

Vers les fonctions d'agrégation

```
SELECT nom, note
FROM Etudiant;
```

-> Chaque ligne affiche **une note individuelle**.

Si on veut connaître **la moyenne de la classe**, ou **le nombre total d'étudiants**,
cette requête suffit-elle ?

👉 SQL peut **calculer** des indicateurs globaux :

- Moyenne des notes (AVG)
- Total des notes (SUM)
- Nombre d'étudiants (COUNT)
- Note la plus haute (MAX)
- Note la plus basse (MIN)

Les fonctions d'agrégation

- AVG (colonne) → moyenne
- COUNT(*) → nombre de lignes
- MIN(colonne), MAX(colonne) → bornes
- SUM(colonne) → somme

Les fonctions d'agrégation

Etudiant

Nom	Prénom	date_n	note	dep	licence
Martin	Véra	85-10-31	13.5	77	2
Martin	Annie	85-12-31	15.5	75	1
Dupont	Sylvie	83-02-03	15.0	77	2
Salma	Sassi	83-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

AVG (colonne) → moyenne

Exemple 1: Moyenne des notes de tous les étudiants

SELECT **AVG(note)** AS moyenne_globale
FROM etudiants;

moyenne-globale
12,24

Les fonctions d'agrégation

Etudiant

Nom	Prénom	date_n	note	dep	licence
Martin	Véra	85-10-31	13.5	77	2
Martin	Annie	85-12-31	15.5	75	1
Dupont	Sylvie	83-02-03	15.0	77	2
Salma	Sassi	83-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

COUNT(*) → nombre de lignes

Exemple 2 – Nombre total d'étudiants

SELECT **COUNT(*)** AS nb_etudiants FROM etudiants;

nb-etudiants
6

Les fonctions d'agrégation

Etudiant

Nom	Prénom	date_n	note	dep	licence
Martin	Véra	85-10-31	13.5	77	2
Martin	Annie	85-12-31	15.5	75	1
Dupont	Sylvie	83-02-03	15.0	77	2
Salma	Sassi	83-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

MIN(colonne), MAX(colonne) → bornes

Exemple 3 – Note la plus haute et la plus basse

SELECT **MIN(note)** AS note_min, **MAX(note)** AS note_max FROM etudiants;

note_min	note_max
5.7	**15.5**

Les fonctions d'agrégation

Etudiant

Nom	Prénom	date_n	note	dep	licence
Martin	Véra	85-10-31	13.5	77	2
Martin	Annie	85-12-31	15.5	75	1
Dupont	Sylvie	83-02-03	15.0	77	2
Salma	Sassi	83-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

SUM (colonne) → somme

Exemple 4: Somme des notes de tous les étudiants

SELECT SUM(note) AS somme_notes
FROM etudiants;

somme_notes
61.2

Autres fonctions d'agrégation

GROUP BY ← Having

GROUP BY

Il s'agit d'une instruction SQL qui va permettre d'utiliser une fonction d'agrégat comme COUNT(), SUM(), sur plusieurs sous groupes de données.

Prérequis

- Afficher des données (SELECT)
- Utiliser des restrictions (WHERE)
- Connaitre les fonctions d'agrégations (SUM, COUNT, AVG, MIN, MAX)

GROUP BY

Etudiant

Nom	Prénom	date_n	note	dep	licence
Martin	Véra	85-10-31	13.5	77	2
Martin	Annie	85-12-31	15.5	75	1
Dupont	Sylvie	83-02-03	15.0	77	2
Salma	Sassi	83-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

Requêtes

1. Compter le nombre total des étudiants (nombre total de lignes).
2. Regrouper les étudiants par département identiques.
3. Pour chaque département, calculer la moyenne des notes.
4. Pour chaque département, calculer le nombre des étudiants et la moyenne des notes.

GROUP BY

1. Compter le nombre total des étudiants (nombre total de lignes)

Etudiant

Nom	Prénom	date_n	note	dep	licence
Martin	Véra	85-10-31	13.5	77	2
Martin	Annie	85-12-31	15.5	75	1
Dupont	Sylvie	83-02-03	15.0	77	2
Salma	Sassi	83-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT COUNT(Nom) AS Total_étudiants
FROM Etudiant;
```

Total_étudiants
6

Fonction **COUNT** sans complexité supplémentaire

GROUP BY

2. Compter le nombre total des étudiants par département identiques

Etudiant

Nom	Prénom	date_n	note	dep	licence
Martin	Véra	85-10-31	13.5	77	2
Martin	Annie	85-12-31	15.5	75	1
Dupont	Sylvie	83-02-03	15.0	77	2
Salma	Sassi	83-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT dep, COUNT(Nom) AS Total_étudiants
FROM Etudiant
Groupe By dep;
```

dep	Total_étudiants
77	2

2 étudiants dans le département 77

GROUP BY permet de regrouper les données

GROUP BY

3. Pour chaque département, calculer la moyenne des notes

Etudiant

Nom	Prénom	date_n	note	dep	licence
Martin	Véra	85-10-31	13.5	77	2
Martin	Annie	85-12-31	15.5	75	1
Dupont	Sylvie	83-02-03	15.0	77	2
Salma	Sassi	83-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT dep, AVG(note) AS moyenne_notes
FROM  Etudiant
Groupe By dep;
```

dep	moyenne_notes
77	14,25

Ajouter une fonction d'agrégation (**AVG**)

GROUP BY

4. Pour chaque département, calculer le nombre des étudiants et la moyenne des notes

Etudiant

Nom	Prénom	date_n	note	dep	licence
Martin	Véra	85-10-31	13.5	77	2
Martin	Annie	85-12-31	15.5	75	1
Dupont	Sylvie	83-02-03	15.0	77	2
Salma	Sassi	83-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT dep, count (nom) AS Total_étudiants,
AVG(note) AS moyenne_notes
FROM Etudiant
Groupe By dep;
```

dep	Total_étudiants	moyenne_notes
77	2	14,25

Combiner des fonctions d'agrégation dans une même requête.

Autres fonctions d'agrégation

GROUP BY ← Having

HAVING

Trés similaire à l'utilisation du « WHERE », le HAVING permet de faire des **restrictions** sur les résultats obtenus avec un « **GROUP BY** »

Prérequis

Savoir utiliser un « GROUP BY »

HAVING

Etudiant

Nom	Prénom	date_n	note	dep	licence
Martin	Véra	85-10-31	13.5	77	2
Martin	Annie	85-12-31	15.5	75	1
Dupont	Sylvie	83-02-03	15.0	77	2
Salma	Sassi	83-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

Requêtes

1. Afficher les départements avec le nombre d'étudiants ayant une note strictement supérieure à 10.

2. Afficher uniquement les départements où la moyenne des notes des étudiants est supérieure à 12.

3. Lister pour chaque département le nombre d'étudiants ayant une note supérieure à 8, et où la moyenne des notes du département est strictement supérieure à 12.

HAVING

1. Afficher les départements avec le nombre d'étudiants ayant une note strictement supérieure à 10

Etudiant

Nom	Prénom	date_n	note	dep	licence
Martin	Véra	85-10-31	13.5	77	2
Martin	Annie	85-12-31	15.5	75	1
Dupont	Sylvie	83-02-03	15.0	77	2
Salma	Sassi	83-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT dep, COUNT(Nom) AS nombre_étudi
ants FROM Etudiant
WHERE note > 10
GROUP BY dep;
```

nombre_étudiants
4

Filtrage avec WHERE (avant GROUP BY)

HAVING

2. Afficher uniquement les départements où la moyenne des notes des étudiants est supérieure à 12

Etudiant

Nom	Prénom	date_n	note	dep	licence
Martin	Véra	85-10-31	13.5	77	2
Martin	Annie	85-12-31	15.5	75	1
Dupont	Sylvie	83-02-03	15.0	77	2
Salma	Sassi	83-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

Filtrer les « moyenne_notes »>12

Groupement des moyennes des notes par département

dep	moyenne_notes
77	14,25
75	15,5
~~93~~	~~5,7~~
~~92~~	~~NULL~~
~~NULL~~	~~11,5~~

dep	moyenne_notes
77	14,25
75	15,5

```
SELECT dep, AVG(note) AS moyenne_notes
FROM Etudiant
GROUP BY dep
HAVING AVG(note) > 12;
```

~~WHERE AVG(note) ???2;~~

HAVING

WHERE *VS* HAVING

3. Lister pour chaque département le nombre d'étudiants ayant tous une note supérieure à 10, et où la moyenne des notes du département est strictement supérieure ou égal à 14,5

Etudiant

Nom	Prénom	date_n	note	dep	licence
Martin	Véra	85-10-31	13.5	77	2
Martin	Annie	85-12-31	15.5	75	1
Dupont	Sylvie	83-02-03	15.0	77	2
Salma	Sassi	83-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

```
SELECT dep, COUNT(nom) AS nombre_étudiants, AVG(note) AS moyenne_notes
FROM Etudiant
WHERE note > 10
GROUP BY dep
HAVING AVG(note) > 14,5;
```

dep	nombre_étudiants	mouyenne_notes
~~77~~	~~2~~	~~14,25~~
75	1	15.5
NULL	1	15.0

dep	nombre_étudiants	mouyenne_notes
75	1	15.5
NULL	1	15.0

GROUP BY &HAVING- Synthèse

- **GROUP BY :** utilisé pour regrouper les données d'une table en sous groupes à l'aide de fonctions d'agrégat.
- **HAVING :** utilisé après un GROUP BY pour effectuer une restriction supplémentaire à l'aide de fonctions d'agrégat.

Syntaxe

```
SELECT colonne1, fonction_aggrégation(colonne2)
FROM table
GROUP BY colonne1
HAVING condition_sur_groupe;
```

Aspect	Group By	HAVING
Quand utiliser ?	Regrouper des données	Filtrer des groupes de données
Application	Avant agrégation	Après agrégation
Filtrage	Agit sur les lignes individuelles	Agit sur les groupes

Au revoir et au prochain cours

Dr. Salma Sassi
salma.sassi@ece.fr